

What Generic Instructions Cost

Measuring the Price of Telling a Model What It Already Knows

Jashwanth Lakshmi Sai Yenugu
Independent Researcher
yenugujashwanth@gmail.com
<https://orcid.org/0009-0008-1831-2615>

September 7, 2026

Abstract

System prompts and agent skills accumulate general advice — write semantic HTML, handle errors, use meaningful names — alongside the project-specific facts a model cannot infer. We measure what that general material costs when the model already knows it. Across 120 runs on two frontier models, we vary only the quantity of general-knowledge padding appended to an otherwise fixed prompt, from 0 to 811 words, and hold the task, the model, and the reasoning effort constant. Padding produced a monotonic **+42% in output tokens and +38% in end-to-end latency** with no observable return. In a LangChain Deep Agents loop, where the prompt is re-sent each turn, the latency penalty reached **+85%** on one model.

We deliberately do not claim that padding degrades output quality, because we could not build an instrument able to detect it. Three pre-registered attempts failed, each for the same reason. Behavioural grading of two build tasks put 47 of 48 outputs at a perfect score, with the single genuine defect occurring at *zero* padding. A rule-following test — five mechanically checkable house rules under the same ladder — found no erosion and had almost no power to find one, since four of the five rules were never broken under any condition. A third attempt commissioned three deliberately hard tasks and calibrated them at zero padding against a pre-registered 55-80% admission band: all three scored 95.5%, 100% and 98.4%, and none was admitted. Deepening the graders from 37 checks to 133 did not move those scores.

We therefore report the harm hypothesis as untested rather than refuted, and regard the ceiling itself as the finding: whatever prompt bloat does to current frontier models, it is not large enough to surface in richly specified single-file front-end builds.

The methodological contribution is the negative space. Six defects were found in our own measuring instruments over the course of this study. The first three each produced a clean, plausible, entirely spurious result, and each was caught only by asking why a number was suspiciously constant; the last three were caught immediately by three controls we should have had from the start — a correct reference implementation the grader must score 100%, an empty stub it must score 0%, and a single-rule mutation that must fail exactly the checks naming that rule. We document all six, because the same failure modes silently underwrite an unknown fraction of published prompt-ablation nulls.

1 The question

A production skill for lesson generation ran to 844 lines. Roughly half of it was material any current frontier model already applies unprompted: use semantic elements, keep headings ordered, label form controls, avoid magic numbers. The other half was genuinely proprietary — the house component names, the required output structure, the project's conventions.

The intuition motivating this work is that the first kind is not free. It consumes context, it is re-sent on every agent turn, and it may compete for attention with the instructions that actually carry information. The question is what it costs, and whether it buys anything.

We separate two claims that are easily conflated:

- **The cost claim.** General-knowledge padding consumes tokens and latency.
- **The harm claim.** General-knowledge padding degrades output.

The first is straightforwardly measurable. The second is the one everybody asserts, and — as we found — the one that is much harder to measure than it looks.

2 Design

Padding ladder. Four cumulative levels of general web-development guidance, fixed in code before any run (`experiments/dose/design.py`):

level	words	content
L0	0	none
L1	68	semantic HTML, heading order, form labels
L2	246	L1 + accessibility, keyboard, contrast
L3	811	L2 + performance, naming, error handling, comments

Every statement is generic best practice, contains no task-specific fact, and is the kind of material that appears in real skills. Nothing in the padding is false, and nothing in it is needed.

Tasks. Two self-contained HTML builds: a five-question quiz with gated submission and full retry reset, and a sortable, filterable employee table with pagination and CSV export. Both are graded behaviourally — the generated page is loaded in jsdom and driven like a user.

Models and harness. `gpt-5.6-luna` and `gpt-5.6-terra`, reasoning effort pinned to `medium` via the Responses API, run through LangChain Deep Agents with a filesystem backend. The agent harness matters: it re-sends the system prompt on every turn, which is precisely the condition under which padding is expected to be expensive.

Scale. 120 runs total, \$9.04: a 16-run direct-API pilot, a 20-run Deep Agents replication, a 48-run dose ladder, a 24-run compliance test, and a 12-run hard-task calibration.

3 What padding costs

Pooled across both tasks and both models, n=12 per dose:

dose	padding words	output tokens	latency
L0	0	3,833	29.1s
L1	68	4,126 (+8%)	31.4s (+8%)
L2	246	4,819 (+26%)	36.6s (+26%)
L3	811	5,428 (+42%)	40.1s (+38%)

Figure 1 plots both. The relationship is monotonic in both columns, and the endpoints are well separated. With 95% bootstrap intervals, L0 output tokens are 3,833 [3,549, 4,105] against L3's 5,428 [5,119, 5,701]; latency is 29.1s [26.6, 31.6] against 40.1s [37.3, 42.9].

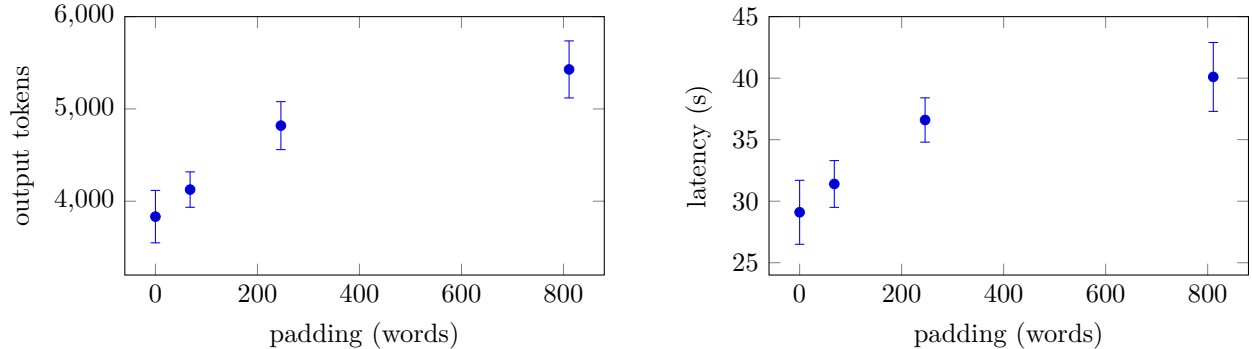


Figure 1: Cost rises monotonically with padding that adds no information. Output tokens (left) and end-to-end latency (right) against the quantity of general-knowledge padding, pooled over two tasks and two models, $n = 12$ per dose. Bars are 95% bootstrap intervals. The padding is *input*, yet it is the output that grows by 42%. Comparing within each task/model/trial cell, the increase holds in all twelve pairs (sign test $p = 0.00024$).

Comparing L0 to L3 *within* each task/model/trial cell rather than across pooled means, the increase holds in every one of the twelve pairs: output tokens +1,595 [+1,241, +1,989], latency +11.0s [+8.7, +13.3], sign test $p = 0.00024$ for both.

The direction is unsurprising; its size and location are not. The padding is 811 words of *input*, yet it is the **output** that grows by 42%. A wordier prompt does not merely cost what it adds — it makes the model answer at greater length, so the bill is paid twice, and the second payment is the larger one at the output token prices both vendors charge.

The agent loop amplifies it. The same minimal-vs-padded contrast, run directly against the API and then through Deep Agents:

harness	model	latency, minimal → padded
direct API	luna	23.4s → 32.9s (+40%)
direct API	terra	17.4s → 26.3s (+51%)
Deep Agents	luna	24.7s → 35.8s (+45%)
Deep Agents	terra	17.9s → 33.2s (+85%)

This is the practically important number. Padding that looks affordable in a single call is re-paid on every turn of an agent loop.

4 What we could not measure

Quality. 47 of 48 dose-ladder outputs passed every behavioural check. The one failure was a genuine model defect — a `RadioNodeList` has no `.closest()` method — and it occurred at **zero** padding, the opposite of the hypothesised direction.

We therefore report the quality result as **unmeasurable, not null**. Both task families sit at ceiling. An instrument that scores 100% without the manipulation cannot show a decrease caused by it, and describing such a result as "no effect on quality" states a conclusion the design cannot support. This distinction is the single most common defect in prompt-ablation reporting, and we decline to add to it.

Rule-following. Because quality was at ceiling, we pre-registered a second test targeting rule *adherence* rather than correctness — the failure mode that motivated the study, where a skill states "never hardcode hex values" twice and the output contains 87 of them. Five mechanically checkable house rules were stated once, before the task, with the same padding ladder appended after.

dose	padding	hex outside :root	other four rules
L0	0	0	0
L1	68	1	1
L2	246	0	0
L3	811	0	0

No erosion. And, again, little power to detect it: four of the five rules were never broken at any dose including zero padding. The one rule with range varied non-monotonically and in opposite directions across the two models.

A third attempt, and a pre-registered failure. Because both instruments were at ceiling, we commissioned three tasks written specifically to be hard, each stating many constraints that must hold simultaneously and producing exact values rather than prose: a formula engine with operator precedence, SUM over ranges, transitive recomputation, cycle detection, #DIV/0, #REF and multi-step undo; a pricing engine with three discount tiers, coupon codes with explicit non-stacking precedence, conditional free shipping and tax on the post-discount subtotal only; and a form with cross-field validation, exact error strings, ordered missing-class reporting and an age boundary.

The admission rule was fixed before any run: a task is carried forward to the ladder only if its pooled score at *zero* padding falls between 55% and 80%.

task	checks	score at zero padding	verdict
sheet	22	95.5%	reject — at ceiling
cart	80	100.0%	reject — at ceiling
form	31	98.4%	reject — at ceiling

No task was admitted, and the band was not moved. Since a perfect score can mean a shallow probe rather than flawless work, we expanded the graders from 37 checks to 133 — boundary values either side of every threshold, error propagation through arithmetic, coupon case-insensitivity, an exactly-18-today date boundary — and re-graded the same twelve files at no cost. The scores did not move. Across twelve runs and 133 checks, only three distinct checks were ever missed.

The honest summary is that we looked for harm three times, with three increasingly sensitive instruments, and found none — while also establishing that no instrument we could build was sensitive enough for that absence to mean much. The harm hypothesis is untested, not refuted. The cost finding does not depend on it.

5 Six measurement bugs, and the controls that stop them

Six defects were found in our own instruments over the course of this study. The first three each produced a clean, plausible result that we believed until we asked why a number looked too tidy. The last three were caught immediately, by controls we should have had from the start.

A constant that was not a measurement. The table task scored exactly 8/10 in all 24 runs — identical across two models and four doses. Two checks were failing deterministically. The `sorts`

check clicked the `<th>`, but every generated file attaches its handler to a `<button data-column>` *inside* the header cell; events propagate up, not down, so sorting never fired. The `active_filter` check compared rows visible on page 1, but the table paginates at 10 per page, so filtering 24 records to 19 leaves 10 rows showing either way — the check could never observe a working filter. Corrected, all 24 files score 10/10. The reported 80% quality ceiling was entirely an artifact.

A rule that scored its own compliant form as a violation. The compliance checker reported hex violations in 17 of 24 files. Two causes, both ours: `<style>` sits inside `<body>`, so summing the two counted every hex twice; and `:root` custom-property definitions were counted as violations when the rule's own second sentence instructs the model to put them there. Corrected: 1 of 24. The rule was also genuinely ambiguous between "never write a hex" and "centralize colours in `:root`", so we score and report both readings rather than silently choosing the flattering one.

Hidden text counted as visible. In an earlier pilot, a correctly hidden results panel still registered as showing a score, because `textContent` includes hidden nodes. This inverted the finding, making padded output look better than minimal.

What finally worked. For the hard-task graders we adopted three controls before grading any model output: a hand-written *correct* reference implementation of each task, which the grader must score 100%; an empty stub page, which it must score 0%; and a single-rule mutation of each reference, which must fail exactly the checks naming that rule and no others. These caught three further defects immediately — `textOf()` returned `""` for a *missing* element, indistinguishable from a valid empty field, so a stub page scored 6/31; `err(k) !== ""` was satisfied by `null`; and our own reference parsed "2008-09-08" with `new Date()`, which reads as UTC midnight and shifts a day earlier under local-time `getDate()`. The check that caught the last one was correct and the reference was wrong, which is exactly the direction a validated instrument is supposed to resolve.

The pattern in the first three bugs is consistent: every one inflated or inverted a result, none produced an obviously broken number, and all three were found by interrogating suspiciously stable values rather than by any test that failed. We report them because a study of this shape is only as good as its instruments, and because the base rate of such bugs in the literature is unknown and probably not low.

6 Related work

A substantial body of work optimizes prompts automatically, searching for wording that scores better on a target task: DSPy (Khattab et al., 2023) compiles declarative model calls into optimized pipelines, GEPA (Agrawal et al., 2025) evolves prompts reflectively, and SPRIG (Zhang et al., 2024), SePO (Tao et al., 2026), SPEAR (Lu et al., 2026) and *p1* (Gao et al., 2026) each search the system-prompt space directly. MAS-PromptBench (Bai and Shi, 2026) asks the adjacent question of when such optimization helps at all in multi-agent settings.

That literature asks what the *best* prompt is. This paper asks a narrower and more mundane question that optimization presupposes: what does prompt content that contributes nothing actually cost? The material we test is not a candidate for optimization, because it is not wrong — it is correct, generic advice the model already follows. Both vendors publish prescriptive prompting guidance of exactly this kind (Anthropic, 2026; OpenAI, 2026), and skills and system prompts accumulate it by imitation.

7 Limitations

Two models from one vendor, one task family (self-contained HTML builds), $n=3$ per cell, and — most importantly — a quality instrument we could not get off ceiling despite three attempts. The cost finding is robust and replicated across four independent runs; the absence of a quality effect is not evidence of absence.

The ceiling appears specific to this task family rather than to the manipulation. Detecting a quality effect, if one exists, likely requires a domain where current models are not already near-perfect — long multi-file refactors, unfamiliar APIs, or tight algorithmic constraints — rather than a harder instance of the same kind of task. That is the obvious next study, and it is a redesign rather than another ladder.

8 What follows for practice

The defensible recommendation is narrow and useful: **general instructions the model already applies are not free**. They cost 42% more output tokens and 38% more latency at the doses we tested, and up to 85% more latency inside an agent loop, for no return we could observe with two different instruments.

That is a sufficient reason to remove them. It does not require the stronger claim that they make the model worse — a claim we tried twice to substantiate and could not, and which we therefore do not make.

References

- Lakshya A Agrawal, Shangyin Tan, Dilara Soyulu, Noah Ziem, et al. GEPA: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2025.
- Anthropic. Prompting best practices. <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices>, 2026. Harvested 2026-08-30; re-diffed 2026-09-04.
- Juyang Bai and Laixi Shi. MAS-PromptBench: When does prompt optimization improve multi-agent llm systems? *arXiv preprint arXiv:2606.23664*, 2026.
- Zhaolin Gao, Yu, Wang, Bo Liu, Thorsten Joachims, Kianté Brantley, and Wen Sun. p1: Better prompt optimization with fewer prompts. *arXiv preprint arXiv:2604.08801*, 2026.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, et al. DSPy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.
- Mengyin Lu, Cong Feng, Huimin Han, Guangming Lu, Yu Sun, Xiaonan Ding, Shihui Long, Fengyi Li, and Tanvi Motwani. SPEAR: Code-augmented agentic prompt optimization. *arXiv preprint arXiv:2605.26275*, 2026.
- OpenAI. Prompt engineering. <https://developers.openai.com/api/docs/guides/prompt-engineering>, 2026. Harvested 2026-08-30; re-diffed 2026-09-04.
- Wangcheng Tao, Han Wu, and Weng-Fai Wong. SePO: Self-evolving prompt agent for system prompt optimization. *arXiv preprint arXiv:2606.04465*, 2026.

Lechen Zhang, Tolga Ergen, Lajanugen Logeswaran, Moontae Lee, and David Jurgens. SPRIG: Improving large language model performance by system prompt optimization. *arXiv preprint arXiv:2410.14826*, 2024.